



GUIA DE EJERCICIOS #1

1. La siguiente actividad será elaborada **solamente en Grupos de 4 compañeros**.
2. A continuación, se describe un conjunto de ejercicios a ser resueltos en Java, aplicando los principios de la programación orientada a objetos.
3. Cada ejercicio debe ser resuelto en un proyecto diferente y ser resuelto de acuerdo con las condiciones especificadas en la descripción de este.
4. Para evitar descuentos generales en la nota de un ejercicio, se debe cumplir cada una de las restricciones brindadas en la descripción correspondiente.
5. Los identificadores dentro del código deben cumplir las reglas de nomenclaturas usadas por las comunidades de desarrolladores de Java.
6. El código fuente elaborado para la solución se debe documentar con un lenguaje técnico "de programador a programador" (por bloques de líneas y líneas específicas, claves en la lógica de la solución) y de manera impersonal.
7. Cada Ejercicio se solucionará en un Proyecto Java distinto, tomando en cuenta los aspectos siguientes:
 - Antes de la definición de su metodo main, incluir como comentario a la lista de TODOS los integrantes del Equipo que entrega la tarea.

- Elaborar un diagrama UML con los paquetes, clases (incluyendo la que tenga al método de inicio main) y asociaciones implementadas en su respectiva solución. El UML debe realizarse con el software Draw.IO. El archivo generado (.drawio) debe exportarse también a un archivo de imagen (.png o .jpg); para luego, guardar ambos archivos en la carpeta de solución del proyecto.
- Comprimir (en .zip o .rar) a la carpeta del proyecto.
- Elegir a un compañero del Equipo para que suba el archivo comprimido del proyecto de solución por el Enlace de envío creado por su docente en el aula digital.

Importante:

- Cada ejercicio resuelto sera enviado por un integrante distinto del Equipo.
- **Cualquier código "extraño" en la solución presentada (que implemente temas aun no vistos en clase de teoría) o algún elemento que genere sospecha de ser resuelto con el uso de IA, hará que El ejercicio sea ignorado.**
- **Si estudiantes no están de acuerdo con la perdida de nota de un Ejercicio (por sospechas brindadas por docente), deberán defender el Ejercicio y responder, explicar perfectamente a TODAS las preguntas que docente considere necesarias para demostrar la autenticidad del código fuente correspondiente y así recuperar la nota correspondiente.**

Ejercicios a Resolver

32% Ejercicio 1: Representacion de una Hora

Java posee varias clases para la gestión de fechas dentro de sus App's, entre ellas a la clase `LocalTime`, usada para representar horas, así como las operaciones que se pueden realizar con ellas.

Por lo tanto, se debe implementar una clase que simule parcialmente el funcionamiento de esta clase `LocalTime`, cumpliendo cuidadosamente a los requerimientos descritos a continuación:

- A. Debe contar con los campos necesarios para almacenar una hora (formada por horas, minutos y segundos), tanto en formato de 12 horas como de 24 horas.
- B. Para simplificar el diseño, la clase solicitada limitara la hora almacenada hasta un segundo menos de 24 horas. Cualquier segundo de mas, hará que la hora almacenada se reinicie al valor: 0:00:00 h para así continuar las operaciones.
- C. Encapsular cada atributo y justificar si requiere a solamente getter o un setter o ambos.
- D. Su método constructor específico inicializa sus campos para registrar la hora exacta de las 12 de la madrugada (12:00:00 am o 0:00h).
- E. Sobrecargar el constructor para aceptar una hora específica en 4 parametros, que incluyen la hora, los minutos, los segundos e indicar si es (am) o (pm). Si la hora recibida no es valida, se deja la inicialización a la hora 12:00:00 am.
- F. Definir otra sobrecarga del constructor para aceptar una hora específica en sus parametros, en formato de 24 horas. Si la hora recibida no es valida, se inicializa a la hora 0:00:00 h.

G. Crear una función que retorne como String a la hora almacenada en sus campos. El usuario podra elegir con un parametro boolean opcional si la desea ver en formato am/pm o 24 horas. Este parametro valdrá true por defecto (indicando que la hora se presentara en formato am/pm).

H. Definir un metodo privado que determine si una hora (en formato 12 horas, recibida en 4 parametros) es correcta o no. Retorna true si es valida.

I. Definir un metodo que permita a usuario actualizar la hora almacenada, enviándola en 4 parametros. Si la hora recibida es valida, se realiza el cambio de la hora interna y retorna (con return) true para confirmar el cambio.

Definir otro metodo similar al descrito en el literal anterior, para actualizar la hora, pero enviada en formato de 24 horas. Si la hora recibida es valida, realiza el cambio de la hora interna y retorna true para confirmar el cambio.

Restricciones:

-Implementar el encapsulamiento apropiado a todos los campos de la clase.

-no esta permitido usar ninguna clase existente de java relacionada al manejo de fechas, de lo contrario, la solución elaborada será descartada.

-Cada método solicitado en el ejercicio y definido en el código, pero que no sea utilizado/invocado, será descartado y se penalizara en la nota de este Ejercicio.

Demostracion:

-Preparar a 3 instancias de la clase modelo para gestión de horas. Para crearlos, utilice un constructor distinto implementado.

-Para confirmar la hora almacenada, por cada objeto, muestre en pantalla a su hora aceptada, tanto en formato am/pm como en 24 h.

-Actualice al objeto 1 con una hora correcta (cualquiera luego del mediodia), al objeto 2 con una hora en formato 24h (cualquiera después de las 8pm) y al ultimo objeto con una fecha 24h incorrecta/invalida.

-Confirmar el cambio de la hora almacenada por cada objeto, mostrando nuevamente en pantalla a su hora aceptada, tanto en formato am/pm como en 24 h.